next-workflow: Architecture Review

An independent review of the Claude Code workflow kit for Next.js: what works, what's risky, and what to improve.

Scope: every file in the kit (8 commands, 7 agents, 3 skills with 17 references, 8 hook scripts, installer, settings, templates, self-test), about 3,000 lines. Reviewed 15 Oct 2026. Findings marked ✓ **VERIFIED** were reproduced by running the code.

1. Verdict

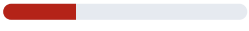
3.4
out of 5

The design is strong, but it needs hardening before a team relies on it.

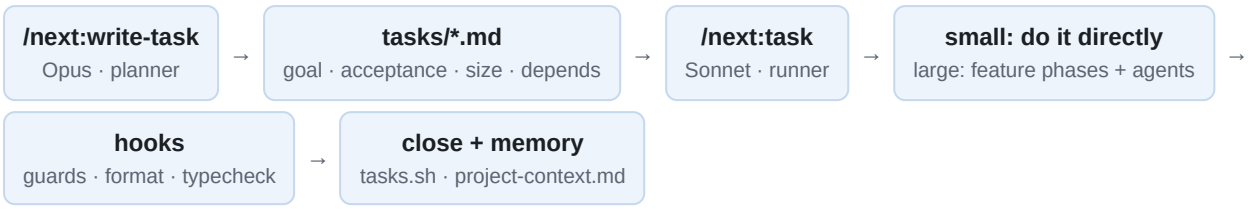
The core idea is right and better than most agent setups. An Opus planner writes specs with testable acceptance criteria. A Sonnet runner builds them. Shell hooks catch Next.js mistakes early, and a memory file keeps each run from re-learning the project. Token discipline is built in throughout.

The weak spots are **security defaults** (two real issues), **recovery** (no per-task isolation or rollback), and **proof** (no measurement of whether the workflow actually produces working features, or what it costs). None of these is hard to fix.

Scorecard

Dimension	Score		Why
Architecture & flow	4.0		Clear plan → build → verify → remember loop, with roles split cleanly.
Token efficiency	4.5		Skills are loaded only when needed, work is routed by size, and verification scales with the change.
Code-quality guardrails	4.0		Catches build-time-only Next errors early; some regex rules are too aggressive.
Security	2.5		Excellent Server Action rules, but the settings and shell code have real holes (H1, H2).
Reliability & recovery	2.5		Batch runs share one context, and there are no checkpoints or rollback.
Verification depth	3.0		Typecheck and build are solid; acceptance is self-graded by reading code.
Framework currency	2.5		Written for Next 15; Next 16 patterns are absent.
Maintainability	3.5		Clean, tested plumbing; no version stamp, README or uninstall.
Observability	1.5		Nothing records cost, time, success rate or guard hits.

2. How the workflow fits together



Around this loop sit `/next:review` (Opus reviewer), `/next:audit` (three auditors in parallel), `/next:test` and `/next:ship`. Rules are supplied by three skills, and deterministic checks run as hooks from `settings.json`, so they apply whatever command is used.

3. The good side

Work is driven by written specs

Every piece of work starts as a task file with a goal, testable acceptance criteria and out-of-scope notes. That's the most important thing for agent quality, and the planner runs on the strongest model.

Memory that compounds

`project-context.md` is injected every session, capped at 4,000 characters, and pruned under explicit rules. Later tasks don't pay to rediscover the codebase.

Deterministic bookkeeping

`tasks.sh` handles status, dependencies, ids and logs. The model never globs or parses task files, which is cheap and leaves no room for it to make bookkeeping mistakes.

The right model for each job

Opus plans, Sonnet builds, and the reviewer is pinned to Opus. Expensive reasoning goes where mistakes cost most (specs and security review).

Catching mistakes early

Hooks catch errors that otherwise only appear at `next build`: async client components, `error.tsx` without `'use client'`, env leaks to the browser, `next/router` in the App Router, zustand singletons, dead Tailwind v4 classes. The Tailwind check reads the installed version, not the declared one.

Security rules for Server Actions

The order authenticate → authorize → validate → change is stated in the data agent, the skill and the reviewer. Treating Server Actions as public endpoints is exactly right.

Token discipline by design

Skills point to separate reference files; tasks are routed by size; feature phases are loaded only for large tasks; verification scales with the change. The runner has explicit rules like "read a file once".

Safe Stop hook

Typecheck runs only when TypeScript actually changed (a dirty marker) and respects `stop_hook_active`, so it can't trap the agent in a loop.

A careful installer

It's namespaced and idempotent, merges settings with a backup, supports dry runs, removes stale commands, and has a 110-assertion self-test that includes regression cases.

One consistent set of opinions

Server-first islands, zod at every boundary and per-request zustand are stated the same way in the skills, agents and guards. The agents don't contradict each other.

4. The bad side: findings

Ordered by severity. HIGH fix before sharing with a team · MEDIUM fix soon · LOW polish.

H1 Settings allow silent reads of everything under /home ✓ VERIFIED

Where `.claude/settings.json:19`: `"Read(//home/**)"`

Impact Claude can read `~/.ssh`, `~/.aws`, and other projects' `.env` files without asking. The `.env` deny rule covers only the current project. Every project the kit is installed into inherits this, and it's Linux-only anyway (macOS uses `/Users`).

Fix Delete the line. Project files are readable by default; nothing in the workflow needs more.

H2 package.json values are run as shell code at session start ✓ VERIFIED

Where `hooks/stack.sh:107-126` writes `KEY="value"` lines without escaping; `session-context.sh:8` and `verify.sh:23` `eval` them. The cache file `.claude/.cache/stack.env` is eval'd too.

Impact A `package.json` with `"next": "$(any command)"` runs that command as soon as a Claude session opens. I reproduced it. With `install.sh --global`, opening any untrusted repo is enough to trigger it.

Fix Emit values with `printf '%s=%q\n'`, or validate each version against `^[0-9A-Za-z.^~<>=|-]*$` before writing it.

H3 All tasks in a batch run in one shared context

Where `commands/next/task.md`, Batch mode: "Loop: next → run that task fully → repeat".

Impact Each task's file reads, tool output and agent reports pile up in one conversation. By task 5–10, context is compacted, earlier details get lost, quality drops and cost per task rises. A mistake in one task can steer the next.

Fix Run each task in a fresh context: a small `run-tasks.sh` loop calling `claude -p "/next:task <id>"` per task, or one subagent per task. The memory file already carries what's needed between tasks; that's what it's for.

H4 No checkpoint or rollback per task

Where The runner and `/next:feature` never touch git; `/next:ship` explicitly doesn't commit.

Impact After `/next:task all`, five tasks' changes are mixed into one diff. If task 3 was wrong, you can't cleanly undo just task 3, and you can't review task by task.

Fix Add an opt-in `NEXT_WF_CHECKPOINT=commit`: one local commit per closed task, with a message like `task 003: <title>`, never pushed. Or run batches in a git worktree and merge it once you're satisfied.

H5 The builder grades its own work

Where `task.md` step 7: "Check the acceptance list item by item" is done by the same model that wrote the code, by reading it.

Impact Criteria like "Empty state shows 'No projects yet'" or "signed-out users are redirected" are never observed running. Self-assessment is the weakest form of verification and tends to say "done" too readily.

Fix (a) Have `/next:write-task` mark each criterion `[test]`, `[browser]` or `[review]`. (b) Add a cheap, independent verifier step: a fresh Sonnet subagent that sees only the task and the diff, plus a Playwright smoke check for `[browser]` items when a dev server is available.

M1 A regex guard blocks valid code ✓ VERIFIED

Where `hooks/guard-component.sh:59` treats any `.get(` within 12 lines of `useEffect(` as data fetching and blocks the edit.

Impact `params.get('tab')`, `map.get(k)` and `formData.get()` inside an effect are all blocked (exit 2). The agent then contorts correct code, or fights the hook and wastes turns.

Fix Match only `fetch(`, `axios.` and known client SDK calls; or turn this rule into a warning. Keep hard blocks for checks that are never wrong (the RSC errors).

M2 Pre-existing type errors fail every turn

Where `hooks/verify.sh:26` typechecks the whole project and fails on any error.

Impact In a codebase that already has type errors, every turn that touches TypeScript is bounced back. The agent either wanders into unrelated fixes (scope creep) or gives up after the one retry the loop guard allows.

Fix Record a baseline of error lines at `/next:setup` or task start, and fail only on *new* errors. Use `tsc --incremental` to keep it fast.

M3 Next.js 16 isn't covered

Where `skills/nextjs-app-router/SKILL.md` says "Assume Next.js 15"; `data-and-caching.md` teaches `unstable_cache`; there's nothing on `'use cache'`, `cacheComponents`, `cacheLife` / `cacheTag` or `proxy.ts`.

Impact On a Next 16 project, which is the current default for new apps, the agent is guided toward deprecated APIs. The stack report knows the version, but nothing changes based on it.

Fix Split the caching reference into `caching-next15.md` and `caching-next16.md`; have the session report say which one to load. Add the Next 16 rules to `pitfalls.md` and the reviewer's checklist.

M4 Three overlapping sources of truth

Where `/next:setup` writes the stack and verify commands into `CLAUDE.md`; `session-context.sh` injects the same facts every session; `project-context.md` holds conventions.

Impact Stack facts load twice per session (wasted tokens), and the `CLAUDE.md` copy goes stale after a dependency upgrade while the hook's copy updates itself. The model sees two different answers.

Fix Make the `CLAUDE.md` block a three-line pointer. Stack facts come only from the hook; conventions only from `project-context.md`.

M5 Files written through Bash skip every guard

Where `settings.json` PostToolUse matcher `Edit|Write|MultiEdit`.

Impact When Claude writes a file with a heredoc or `sed` (common in auto mode), there's no RSC guard, component guard, formatting or dirty marker, so the Stop typecheck is skipped too.

Fix State in `task.md` and the agents: "edit source files only with Edit/Write". Back it up with a cheap PostToolUse(Bash) hook that touches the dirty marker when `git status` shows changed `.ts/.tsx` files.

M6 Nothing tests the agent's behaviour, only the plumbing

Where `test.sh`: 110 assertions on hooks, installer and `tasks.sh`.

Impact A change to a prompt (a command, agent or skill) can make outcomes worse without anything failing. You can't tell whether a change helped.

Fix Build a small eval: a fixture Next app, 5–8 seeded tasks of different sizes, run headless with `claude -p`, then score acceptance pass rate, build success, tokens and wall time. Run it before and after any prompt change.

M7 No numbers on cost, time or success

Where The task log records one line of prose; the hooks keep no record of what they caught.

Impact You can't answer "is Sonnet good enough?", "which guard fires most?" or "what does a large task cost?". The model split you just made can't be tuned without data.

Fix Have the hooks append one JSON line per event to `.claude/.cache/metrics.jsonl` (guard name, file, block or warn). Have the runner log duration, files changed, retries and size per task. A `tasks.sh stats` command can summarise both.

L1 "Read-only" auditors can still write

Where `next-reviewer`, `next-perf-auditor` and `next-a11y-auditor` list `Bash` among their tools.

Fix "Read-only" is only an instruction; Bash can write files. Keep Bash only where the agent needs it (perf runs the build), and say so in that agent's file.

L2 Small contradictions in the prompts

Where `next-reviewer.md` items 8 and 9 overlap; `next-ui-builder.md` says to run `node -p` for the Tailwind version, which the session report already gives; `next-test-writer.md` says to "set up the minimum" test config, which conflicts with "never add a dependency unasked".

Fix Merge the reviewer items, point to the session report, and have the test writer *ask* before adding test tooling.

L3 Each edit starts several processes

Where Four PostToolUse hooks per edit, each re-parsing JSON; `guard-styles.sh:21` starts Node on every edit to read the Tailwind version instead of using the stack cache; `eslint --fix` runs on every edit.

Fix Use one dispatcher hook that parses the input once and reuses `stack.env`. Leave ESLint to the Stop hook or to `/next:ship` on large repos.

L4 **The .env protection is partial**

Where The `deny` rules cover the Read tool only; `cat .env` through Bash isn't blocked.

Fix Add `Bash(cat .env*)`-style deny patterns, or a PreToolUse hook that rejects Bash commands mentioning `.env` or `.pem`.

L5 **No version stamp, README or uninstall**

Where The installer doesn't record which kit version a project has.

Fix Ship a `VERSION` file, copy it to `.claude/next-workflow.version`, show it in the session report, add `install.sh --uninstall`, and write the README that's still open.

L6 **No review step between planning and building**

Where `/next:write-task` writes `status: todo`, so `/next:task all` can start building immediately.

Fix Have the planner write `status: draft`, and add `tasks.sh approve <id|all>`. That way the plan is reviewed before any money is spent building it.

5. Improvement roadmap

NOW **Quick wins: about one day**

- **H1**: remove `Read(//home/**)` from settings.
- **H2**: escape the stack values (`%q`) or validate versions before eval.
- **M1**: narrow the effect-fetch rule, or make it a warning.
- **M5**: add the "edit only with Edit/Write" rule, and set the dirty marker after Bash.
- **L2, L4**: prompt cleanups and the .env Bash deny.

NEXT **Reliability: about one week**

- **H3**: a fresh context per task (a headless runner loop or one subagent per task).
- **H4**: an opt-in commit per closed task, or worktree batches.
- **H5**: tagged acceptance criteria, an independent verifier subagent, and a Playwright smoke check.
- **M2**: a type-error baseline, failing only on new errors.
- **M4**: one source of truth for stack facts.
- **L6**: draft → approve gate for planned tasks.

LATER **Strategic: ongoing**

- **M3**: version-aware Next 15/16 references, chosen by the session report.
- **M6**: an eval harness with a fixture app and seeded tasks, run on every prompt change.
- **M7**: metrics in a JSONL file, plus `tasks.sh stats`. Use the data to decide when Haiku is good enough.
- **L3, L5**: a single hook dispatcher, version stamp, uninstall and README.

If you do only three things: fix H1 and H2 (security, under an hour together), and give each task its own fresh context (H3). Those three changes do the most for safety and output quality for the least work.

6. Appendix: how the verified findings were reproduced

M1: guard blocks a valid component

```
'use client'
export function Tabs() {
  const params = useSearchParams()
  const [tab, setTab] = useState('a')
  useEffect(() => { setTab(params.get('tab') ?? 'a') }, [params])
  ...
→ guard-component.sh: "✗ Data fetching inside useEffect"   exit=2
```

H2: command runs from package.json

```
{"dependencies":{"next":"$(echo INJECTED-RAN >&2)"}}
→ session-context.sh prints: INJECTED-RAN
```

H1: broad read permission

```
.claude/settings.json:19    "Read(//home/**)"
```